Sentiment Analysis using Transformer Models – Hugging Face

2403A52006

MD.Mustafa

Install Required Libraries

```
"""
STEP 1: Install Required Libraries
"""

!pip install transformers datasets torch pandas scikit-learn -q
!pip install --upgrade transformers datasets accelerate -q
```

Import Libraries

```
"""
STEP 2: Import Required Libraries
"""

import torch
import numpy as np
import pandas as pd

from datasets import load_dataset

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix

import seaborn as sns
import matplotlib.pyplot as plt
```

Load Dataset

```
"""
STEP 3: Load IMDB Sentiment Dataset
"""

dataset = load_dataset("imdb")

print(dataset)

print("\nExample review:\n")
print(dataset["train"][0])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

```
Example review:

{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fir
```

Check Class Distribution

```
"""
STEP 4: Analyze Class Distribution
"""

labels = dataset["train"]["label"]

unique, counts = np.unique(labels, return_counts=True)

print("Class Distribution:")
for u, c in zip(unique, counts):
    print(f"Label {u}:", c)
```

```
Class Distribution:
Label 0: 12500
Label 1: 12500
```

Load Transformer Tokenizer

```
"""
STEP 5: Load BERT Tokenizer
"""

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

Tokenization Function

```
"""
STEP 6: Tokenize Text Data
"""

def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```
Map: 100%  25000/25000 [00:39<00:00, 525.94 examples/s]
Map: 100%  25000/25000 [00:26<00:00, 1233.40 examples/s]
Map: 100%  50000/50000 [00:43<00:00, 1236.69 examples/s]
```

Prepare Dataset for PyTorch

```
"""
STEP 7: Convert Dataset to PyTorch Format
"""

tokenized_dataset = tokenized_dataset.remove_columns(["text"])
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")

tokenized_dataset.set_format("torch")

train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]
```

Load Pretrained Transformer Model

```
"""
STEP 8: Load Pretrained BERT Model
"""
```

```
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
Loading weights: 100%                                                    199/199  [00:00<00:00, 4494.63it/s]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
classifier.bias                             | MISSING    |
classifier.weight                           | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

Define Evaluation Metrics

```
"""
STEP 9: Define Evaluation Metrics
"""

def compute_metrics(eval_pred):
    logits, labels = eval_pred

    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels,
        predictions,
        average="binary"
    )

    accuracy = accuracy_score(labels, predictions)

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

Training Configuration

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=1,
    weight_decay=0.01,
    logging_dir="./logs"
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
```

Initialize Trainer

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

Train the Model

```
"""
STEP 12: Train the Model
"""

trainer.train()
```

[1563/1563 13:22, Epoch 1/1]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.289846	0.271260	0.886920	0.882241	0.893040	0.887608

```
Writing model shards: 100%                                    1/1 [00:06<00:00,  6.67s/it]
Writing model shards: 100%                                    1/1 [00:05<00:00,  5.06s/it]
Writing model shards: 100%                                    1/1 [00:10<00:00, 10.99s/it]
Writing model shards: 100%                                    1/1 [00:05<00:00,  5.14s/it]
TrainOutput(global_step=1563, training_loss=0.32844362850762754, metrics={'train_runtime': 803.9005,
'train_samples_per_second': 31.098, 'train_steps_per_second': 1.944, 'total_flos': 1644444096000000.0, 'train_loss':
0.32844362850762754, 'epoch': 1.0})
```

Evaluate Model

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

results = trainer.evaluate()

print("\nEvaluation Results:\n")
print(results)
```

[1563/1563 03:06]

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
/tmp/ipykernel_4396/320694979.py in <cell line: 0>()
      7 )
      8
----> 9 results = trainer.evaluate()
     10
     11 print("\nEvaluation Results:\n")

                              4 frames
/usr/local/lib/python3.12/dist-packages/transformers/utils/notebook.py in _require(x, msg)
     29 def _require(x: _T | None, msg: str) -> _T:
     30     if x is None:
---> 31         raise RuntimeError(msg)
     32     return x
     33

RuntimeError: on_train_begin must be called before on_evaluate
```